

Table of Contents

Algebra	2
Dynamic Programming	33
Graphs	40
Geometry	43
Numerical Methods	49

Translated

From: e-maxx.ru

Binary Exponentiation

Binary exponentiation (also known as exponentiation by squaring) is a trick which allows to calculate a^n using only $O(\log n)$ multiplications (instead of $O(n)$ multiplications required by the naive approach).

It also has important applications in many tasks unrelated to arithmetic, since it can be used with any operations that have the property of **associativity**:

$$(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$$

Most obviously this applies to modular multiplication, to multiplication of matrices and to other problems which we will discuss below.

Algorithm

Raising a to the power of n is expressed naively as multiplication by a done $n - 1$ times: $a^n = a \cdot a \cdot \dots \cdot a$. However, this approach is not practical for large a or n .

$$a^{b+c} = a^b \cdot a^c \text{ and } a^{2b} = a^b \cdot a^b = (a^b)^2.$$

The idea of binary exponentiation is, that we split the work using the binary representation of the exponent.

Let's write n in base 2, for example:

$$3^{13} = 3^{1101_2} = 3^8 \cdot 3^4 \cdot 3^1$$

Since the number n has exactly $\lfloor \log_2 n \rfloor + 1$ digits in base 2, we only need to perform $O(\log n)$ multiplications, if we know the powers $a^1, a^2, a^4, a^8, \dots, a^{2^{\lfloor \log n \rfloor}}$.

So we only need to know a fast way to compute those. Luckily this is very easy, since an element in the sequence is just the square of the previous element.

$$3^1 = 3$$

$$3^2 = (3^1)^2 = 3^2 = 9$$

$$3^4 = (3^2)^2 = 9^2 = 81$$

$$3^8 = (3^4)^2 = 81^2 = 6561$$

So to get the final answer for 3^{13} , we only need to multiply three of them (skipping 3^2 because the corresponding bit in n is not set): $3^{13} = 6561 \cdot 81 \cdot 3 = 1594323$

The final complexity of this algorithm is $O(\log n)$: we have to compute $\log n$ powers of a , and then have to do at most $\log n$ multiplications to get the final answer from them.

The following recursive approach expresses the same idea:

$$a^n = \begin{cases} 1 & \text{if } n == 0 \\ (a^{\frac{n}{2}})^2 & \text{if } n > 0 \text{ and } n \text{ even} \\ (a^{\frac{n-1}{2}})^2 \cdot a & \text{if } n > 0 \text{ and } n \text{ odd} \end{cases}$$

Implementation

First the recursive approach, which is a direct translation of the recursive formula:

```
long long binpow(long long a, long long b) {
    if (b == 0)
        return 1;
    long long res = binpow(a, b / 2);
    if (b % 2)
        return res * res * a;
    else
        return res * res;
}
```

The second approach accomplishes the same task without recursion. It computes all the powers in a loop, and multiplies the ones with the corresponding set bit in n . Although the complexity of both approaches is identical, this approach will be faster in practice since we don't have the overhead of the recursive calls.

```
long long binpow(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}
```

Applications

Effective computation of large exponents modulo a number

Problem: Compute $x^n \bmod m$. This is a very common operation. For instance it is used in computing the [modular multiplicative inverse](#).

Solution: Since we know that the modulo operator doesn't interfere with multiplications ($a \cdot b \equiv (a \bmod m) \cdot (b \bmod m) \pmod m$), we can directly use the same code, and just replace every multiplication with a modular multiplication:

```
long long binpow(long long a, long long b, long long m) {
    a %= m;
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}
```

Note: It's possible to speed this algorithm for large $b \gg m$. If m is a prime number $x^n \equiv x^{n \bmod (m-1)} \pmod m$ for prime m , and $x^n \equiv x^{n \bmod \phi(m)} \pmod m$ for

composite m . This follows directly from Fermat's little theorem and Euler's theorem, see the article about [Modular Inverses](#) for more details.

Effective computation of Fibonacci numbers

Problem: Compute n -th Fibonacci number F_n .

Solution: For more details, see the [Fibonacci Number article](#). We will only go through an overview of the algorithm. To compute the next Fibonacci number, only the two previous ones are needed, as $F_n = F_{n-1} + F_{n-2}$. We can build a 2×2 matrix that describes this transformation: the transition from F_i and F_{i+1} to F_{i+1} and F_{i+2} . For example, applying this transformation to the pair F_0 and F_1 would change it into F_1 and F_2 . Therefore, we can raise this transformation matrix to the n -th power to find F_n in time complexity $O(\log n)$.

Applying a permutation k times

Problem: You are given a sequence of length n . Apply to it a given permutation k times.

Solution: Simply raise the permutation to k -th power using binary exponentiation, and then apply it to the sequence. This will give you a time complexity of $O(n \log k)$.

```
vector<int> applyPermutation(vector<int> sequence, vector<int>
permutation) {
    vector<int> newSequence(sequence.size());
    for(int i = 0; i < sequence.size(); i++) {
        newSequence[i] = sequence[permutation[i]];
    }
    return newSequence;
}

vector<int> permute(vector<int> sequence, vector<int> permutation, long
long k) {
    while (k > 0) {
        if (k & 1) {
            sequence = applyPermutation(sequence, permutation);
        }
        permutation = applyPermutation(permutation, permutation);
        k >>= 1;
    }
}
```

```
    return sequence;
}
```

Note: This task can be solved more efficiently in linear time by building the permutation graph and considering each cycle independently. You could then compute k modulo the size of the cycle and find the final position for each number which is part of this cycle.

Fast application of a set of geometric operations to a set of points

Problem: Given n points p_i , apply m transformations to each of these points. Each transformation can be a shift, a scaling or a rotation around a given axis by a given angle. There is also a "loop" operation which applies a given list of transformations k times ("loop" operations can be nested). You should apply all transformations faster than $O(n \cdot \text{length})$, where length is the total number of transformations to be applied (after unrolling "loop" operations).

Solution: Let's look at how the different types of transformations change the coordinates:

- Shift operation: adds a different constant to each of the coordinates.
- Scaling operation: multiplies each of the coordinates by a different constant.
- Rotation operation: the transformation is more complicated (we won't go in details here), but each of the new coordinates still can be represented as a linear combination of the old ones.

As you can see, each of the transformations can be represented as a linear operation on the coordinates. Thus, a transformation can be written as a 4×4 matrix of the form:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix}$$

that, when multiplied by a vector with the old coordinates and a unit gives a new vector with the new coordinates and a unit:

$$(x \quad y \quad z \quad 1) \cdot \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} = (x' \quad y' \quad z' \quad 1)$$

(Why introduce a fictitious fourth coordinate, you ask? That is the beauty of [homogeneous coordinates](#), which find great application in computer graphics. Without this, it would not be possible to implement affine operations like the shift operation as a single matrix multiplication, as it requires us to *add* a constant to the coordinates. The affine transformation becomes a linear transformation in the higher dimension!)

Here are some examples of how transformations are represented in matrix form:

- Shift operation: shift x coordinate by 5, y coordinate by 7 and z coordinate by 9.

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 5 & 7 & 9 & 1 \end{pmatrix}$$

- Scaling operation: scale the x coordinate by 10 and the other two by 5.

$$\begin{pmatrix} 10 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Rotation operation: rotate θ degrees around the x axis following the right-hand rule (counter-clockwise direction).

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Now, once every transformation is described as a matrix, the sequence of transformations can be described as a product of these matrices, and a "loop" of k repetitions can be described as the matrix raised to the power of k (which can be calculated using binary exponentiation in $O(\log k)$). This way, the matrix which represents all transformations can be calculated first in $O(m \log k)$, and then it can be applied to each of the n points in $O(n)$ for a total complexity of $O(n + m \log k)$.

Number of paths of length k in a graph

Problem: Given a directed unweighted graph of n vertices, find the number of paths of length k from any vertex u to any other vertex v .

Solution: This problem is considered in more detail in [a separate article](#). The algorithm consists of raising the adjacency matrix M of the graph (a matrix where $m_{ij} = 1$ if there is an edge from i to j , or 0 otherwise) to the k -th power. Now m_{ij} will be the number of paths of length k from i to j . The time complexity of this solution is $O(n^3 \log k)$.

Note: In that same article, another variation of this problem is considered: when the edges are weighted and it is required to find the minimum weight path containing exactly k edges. As shown in that article, this problem is also solved by exponentiation of the adjacency matrix. The matrix would have the weight of the edge from i to j , or ∞ if there is no such edge. Instead of the usual operation of multiplying two matrices, a modified one should be used: instead of multiplication, both values are added, and instead of a summation, a minimum is taken. That is: $\text{result}_{ij} = \min_{1 \leq k \leq n} (a_{ik} + b_{kj})$.

Variation of binary exponentiation: multiplying two numbers modulo m

Problem: Multiply two numbers a and b modulo m . a and b fit in the built-in data types, but their product is too big to fit in a 64-bit integer. The idea is to compute $a \cdot b \pmod{m}$ without using bignum arithmetics.

Solution: We simply apply the binary construction algorithm described above, only performing additions instead of multiplications. In other words, we have "expanded" the multiplication of two numbers to $O(\log m)$ operations of addition and multiplication by two (which, in essence, is an addition).

$$a \cdot b = \begin{cases} 0 & \text{if } a = 0 \\ 2 \cdot \frac{a}{2} \cdot b & \text{if } a > 0 \text{ and } a \text{ even} \\ 2 \cdot \frac{a-1}{2} \cdot b + b & \text{if } a > 0 \text{ and } a \text{ odd} \end{cases}$$

Note: You can solve this task in a different way by using floating-point operations. First compute the expression $\frac{a \cdot b}{m}$ using floating-point numbers and cast it to an unsigned integer q . Subtract $q \cdot m$ from $a \cdot b$ using unsigned integer arithmetics and take it modulo m to find the answer. This solution looks rather unreliable, but it is very fast, and very easy to implement. See [here](#) for more information.

Translated

From: e-maxx.ru

Euclidean algorithm for computing the greatest common divisor

Given two non-negative integers a and b , we have to find their **GCD** (greatest common divisor), i.e. the largest number which is a divisor of both a and b . It's commonly denoted by $\gcd(a, b)$. Mathematically it is defined as:

$$\gcd(a, b) = \max \{k > 0 : (k \mid a) \text{ and } (k \mid b)\}$$

(here the symbol " $\mid$ " denotes divisibility, i.e. " $k \mid a$ " means " k divides a ")

When one of the numbers is zero, while the other is non-zero, their greatest common divisor, by definition, is the second number. When both numbers are zero, their greatest common divisor is undefined (it can be any arbitrarily large number), but it is convenient to define it as zero as well to preserve the associativity of $\gcd$. Which gives us a simple rule: if one of the numbers is zero, the greatest common divisor is the other number.

The Euclidean algorithm, discussed below, allows to find the greatest common divisor of two numbers a and b in $O(\log \min(a, b))$. Since the function is **associative**, to find the GCD of **more than two numbers**, we can do $\gcd(a, b, c) = \gcd(a, \gcd(b, c))$ and so forth.

The algorithm was first described in Euclid's "Elements" (circa 300 BC), but it is possible that the algorithm has even earlier origins.

Algorithm

Originally, the Euclidean algorithm was formulated as follows: subtract the smaller number from the larger one until one of the numbers is zero. Indeed, if g divides a and b , it also divides $a - b$. On the other hand, if g divides $a - b$ and b , then it also divides

$a = b + (a - b)$, which means that the sets of the common divisors of $\{a, b\}$ and $\{b, a - b\}$ coincide.

Note that a remains the larger number until b is subtracted from it at least $\lfloor \frac{a}{b} \rfloor$ times.

Therefore, to speed things up, $a - b$ is substituted with $a - \lfloor \frac{a}{b} \rfloor b = a \bmod b$. Then the algorithm is formulated in an extremely simple way:

$$\text{gcd}(a, b) = \begin{cases} a, & \text{if } b = 0 \\ \text{gcd}(b, a \bmod b), & \text{otherwise.} \end{cases}$$

Implementation

```
int gcd (int a, int b) {
    if (b == 0)
        return a;
    else
        return gcd (b, a % b);
}
```

Using the ternary operator in C++, we can write it as a one-liner.

```
int gcd (int a, int b) {
    return b ? gcd (b, a % b) : a;
}
```

And finally, here is a non-recursive implementation:

```
int gcd (int a, int b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}
```

Note that since C++17, `gcd` is implemented as a [standard function](#) in C++.

Time Complexity

The running time of the algorithm is estimated by Lamé's theorem, which establishes a surprising connection between the Euclidean algorithm and the Fibonacci sequence:

If $a > b \geq 1$ and $b < F_n$ for some n , the Euclidean algorithm performs at most $n - 2$ recursive calls.

Moreover, it is possible to show that the upper bound of this theorem is optimal. When $a = F_n$ and $b = F_{n-1}$, $\gcd(a, b)$ will perform exactly $n - 2$ recursive calls. In other words, consecutive Fibonacci numbers are the worst case input for Euclid's algorithm.

Given that Fibonacci numbers grow exponentially, we get that the Euclidean algorithm works in $O(\log \min(a, b))$.

Another way to estimate the complexity is to notice that $a \bmod b$ for the case $a \geq b$ is at least 2 times smaller than a , so the larger number is reduced at least in half on each iteration of the algorithm. Applying this reasoning to the case when we compute the GCD of the set of numbers $a_1, \dots, a_n \leq C$, this also allows us to estimate the total runtime as $O(n + \log C)$, rather than $O(n \log C)$, since every non-trivial iteration of the algorithm reduces the current GCD candidate by at least a factor of 2.

Least common multiple

Calculating the least common multiple (commonly denoted **LCM**) can be reduced to calculating the GCD with the following simple formula:

$$\text{lcm}(a, b) = \frac{a \cdot b}{\gcd(a, b)}$$

Thus, LCM can be calculated using the Euclidean algorithm with the same time complexity:

A possible implementation, that cleverly avoids integer overflows by first dividing a with the GCD, is given here:

```
int lcm (int a, int b) {
    return a / gcd(a, b) * b;
}
```

Binary GCD

The Binary GCD algorithm is an optimization to the normal Euclidean algorithm.

The slow part of the normal algorithm are the modulo operations. Modulo operations, although we see them as $O(1)$, are a lot slower than simpler operations like addition, subtraction or bitwise operations. So it would be better to avoid those.

It turns out, that you can design a fast GCD algorithm that avoids modulo operations. It's based on a few properties:

- If both numbers are even, then we can factor out a two of both and compute the GCD of the remaining numbers: $\text{gcd}(2a, 2b) = 2 \text{gcd}(a, b)$.
- If one of the numbers is even and the other one is odd, then we can remove the factor 2 from the even one: $\text{gcd}(2a, b) = \text{gcd}(a, b)$ if b is odd.
- If both numbers are odd, then subtracting one number of the other one will not change the GCD: $\text{gcd}(a, b) = \text{gcd}(b, a - b)$

Using only these properties, and some fast bitwise functions from GCC, we can implement a fast version:

```
int gcd(int a, int b) {
    if (!a || !b)
        return a | b;
    unsigned shift = __builtin_ctz(a | b);
    a >>= __builtin_ctz(a);
    do {
        b >>= __builtin_ctz(b);
        if (a > b)
            swap(a, b);
        b -= a;
    } while (b);
    return a << shift;
}
```

Notice, that such an optimization is usually not necessary, and most programming languages already have a GCD function in their standard libraries. E.g. C++17 has such a function `std::gcd` in the `numeric` header.

Translated

From: e-maxx.ru

Extended Euclidean Algorithm

While the [Euclidean algorithm](#) calculates only the greatest common divisor (GCD) of two integers a and b , the extended version also finds a way to represent GCD in terms of a and b , i.e. coefficients x and y for which:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

It's important to note that by [Bézout's identity](#) we can always find such a representation. For instance, $\gcd(55, 80) = 5$, therefore we can represent 5 as a linear combination with the terms 55 and 80: $55 \cdot 3 + 80 \cdot (-2) = 5$

A more general form of that problem is discussed in the article about [Linear Diophantine Equations](#). It will build upon this algorithm.

Algorithm

We will denote the GCD of a and b with g in this section.

The changes to the original algorithm are very simple. If we recall the algorithm, we can see that the algorithm ends with $b = 0$ and $a = g$. For these parameters we can easily find coefficients, namely $g \cdot 1 + 0 \cdot 0 = g$.

Starting from these coefficients $(x, y) = (1, 0)$, we can go backwards up the recursive calls. All we need to do is to figure out how the coefficients x and y change during the transition from (a, b) to $(b, a \bmod b)$.

Let us assume we found the coefficients (x_1, y_1) for $(b, a \bmod b)$:

$$b \cdot x_1 + (a \bmod b) \cdot y_1 = g$$

and we want to find the pair (x, y) for (a, b) :

$$a \cdot x + b \cdot y = g$$

We can represent $a \bmod b$ as:

$$a \bmod b = a - \lfloor \frac{a}{b} \rfloor \cdot b$$

Substituting this expression in the coefficient equation of (x_1, y_1) gives:

$$g = b \cdot x_1 + (a \bmod b) \cdot y_1 = b \cdot x_1 + (a - \lfloor \frac{a}{b} \rfloor \cdot b) \cdot y_1$$

and after rearranging the terms:

$$g = a \cdot y_1 + b \cdot (x_1 - y_1 \cdot \lfloor \frac{a}{b} \rfloor)$$

We found the values of x and y :

$$\begin{cases} x = y_1 \\ y = x_1 - y_1 \cdot \lfloor \frac{a}{b} \rfloor \end{cases}$$

Implementation

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
```

```

        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

```

The recursive function above returns the GCD and the values of coefficients to `x` and `y` (which are passed by reference to the function).

This implementation of extended Euclidean algorithm produces correct results for negative integers as well.

Iterative version

It's also possible to write the Extended Euclidean algorithm in an iterative way. Because it avoids recursion, the code will run a little bit faster than the recursive one.

```

int gcd(int a, int b, int& x, int& y) {
    x = 1, y = 0;
    int x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1) {
        int q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x - q * x1);
        tie(y, y1) = make_tuple(y1, y - q * y1);
        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
    }
    return a1;
}

```

If you look closely at the variables `a1` and `b1`, you can notice that they take exactly the same values as in the iterative version of the normal [Euclidean algorithm](#). So the algorithm will at least compute the correct GCD.

To see why the algorithm computes the correct coefficients, consider that the following invariants hold at any given time (before the while loop begins and at the end of each iteration):

$$x \cdot a + y \cdot b = a_1$$

$$x_1 \cdot a + y_1 \cdot b = b_1$$

Let the values at the end of an iteration be denoted by a prime ('), and assume $q = \frac{a_1}{b_1}$.
From the [Euclidean algorithm](#), we have:

$$a'_1 = b_1$$

$$b'_1 = a_1 - q \cdot b_1$$

For the first invariant to hold, the following should be true:

$$x' \cdot a + y' \cdot b = a'_1 = b_1$$

$$x' \cdot a + y' \cdot b = x_1 \cdot a + y_1 \cdot b$$

Similarly for the second invariant, the following should hold:

$$x'_1 \cdot a + y'_1 \cdot b = a_1 - q \cdot b_1$$

$$x'_1 \cdot a + y'_1 \cdot b = (x - q \cdot x_1) \cdot a + (y - q \cdot y_1) \cdot b$$

By comparing the coefficients of a and b , the update equations for each variable can be derived, ensuring that the invariants are maintained throughout the algorithm.

At the end we know that a_1 contains the GCD, so $x \cdot a + y \cdot b = g$. Which means that we have found the required coefficients.

You can even optimize the code more, and remove the variable a_1 and b_1 from the code, and just reuse a and b . However if you do so, you lose the ability to argue about the invariants.

Translated

From: e-maxx.ru

Linear Diophantine Equation

A Linear Diophantine Equation (in two variables) is an equation of the general form:

$$ax + by = c$$

where a, b, c are given integers, and x, y are unknown integers.

In this article, we consider several classical problems on these equations:

- finding one solution
- finding all solutions
- finding the number of solutions and the solutions themselves in a given interval
- finding a solution with minimum value of $x + y$

The degenerate case

A degenerate case that need to be taken care of is when $a = b = 0$. It is easy to see that we either have no solutions or infinitely many solutions, depending on whether $c = 0$ or not. In the rest of this article, we will ignore this case.

Analytic solution

When $a \neq 0$ and $b \neq 0$, the equation $ax + by = c$ can be equivalently treated as either of the following:

$$ax \equiv c \pmod{b}$$

$$by \equiv c \pmod{a}$$

Without loss of generality, assume that $b \neq 0$ and consider the first equation. When a and b are co-prime, the solution to it is given as

$$x \equiv ca^{-1} \pmod{b},$$

where a^{-1} is the [modular inverse](#) of a modulo b .

When a and b are not co-prime, values of ax modulo b for all integer x are divisible by $g = \gcd(a, b)$, so the solution only exists when c is divisible by g . In this case, one of solutions can be found by reducing the equation by g :

$$(a/g)x \equiv (c/g) \pmod{b/g}.$$

By the definition of g , the numbers a/g and b/g are co-prime, so the solution is given explicitly as

$$\begin{cases} x \equiv (c/g)(a/g)^{-1} \pmod{b/g}, \\ y = \frac{c-ax}{b}. \end{cases}$$

Algorithmic solution

Bézout's lemma (also called Bézout's identity) is a useful result that can be used to understand the following solution.

Let $g = \gcd(a, b)$. Then there exist integers x, y such that $ax + by = g$.

Moreover, g is the least such positive integer that can be written as $ax + by$; all integers of the form $ax + by$ are multiples of g .

To find one solution of the Diophantine equation with 2 unknowns, you can use the [Extended Euclidean algorithm](#). First, assume that a and b are non-negative. When we apply Extended Euclidean algorithm for a and b , we can find their greatest common divisor g and 2 numbers x_g and y_g such that:

$$ax_g + by_g = g$$

If c is divisible by $g = \gcd(a, b)$, then the given Diophantine equation has a solution, otherwise it does not have any solution. The proof is straight-forward: a linear combination of two numbers is divisible by their common divisor.

Now suppose that c is divisible by g , then we have:

$$a \cdot x_g \cdot \frac{c}{g} + b \cdot y_g \cdot \frac{c}{g} = c$$

Therefore one of the solutions of the Diophantine equation is:

$$x_0 = x_g \cdot \frac{c}{g},$$

$$y_0 = y_g \cdot \frac{c}{g}.$$

The above idea still works when a or b or both of them are negative. We only need to change the sign of x_0 and y_0 when necessary.

Finally, we can implement this idea as follows (note that this code does not consider the case $a = b = 0$):

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(int a, int b, int c, int &x0, int &y0, int &g) {
```

```

g = gcd(abs(a), abs(b), x0, y0);
if (c % g) {
    return false;
}

x0 *= c / g;
y0 *= c / g;
if (a < 0) x0 = -x0;
if (b < 0) y0 = -y0;
return true;
}

```

Getting all solutions

From one solution (x_0, y_0) , we can obtain all the solutions of the given equation.

Let $g = \gcd(a, b)$ and let x_0, y_0 be integers which satisfy the following:

$$a \cdot x_0 + b \cdot y_0 = c$$

Now, we should see that adding b/g to x_0 , and, at the same time subtracting a/g from y_0 will not break the equality:

$$a \cdot \left(x_0 + \frac{b}{g}\right) + b \cdot \left(y_0 - \frac{a}{g}\right) = a \cdot x_0 + b \cdot y_0 + a \cdot \frac{b}{g} - b \cdot \frac{a}{g} = c$$

Obviously, this process can be repeated again, so all the numbers of the form:

$$x = x_0 + k \cdot \frac{b}{g}$$

$$y = y_0 - k \cdot \frac{a}{g}$$

are solutions of the given Diophantine equation.

Since the equation is linear, all solutions lie on the same line, and by the definition of g this is the set of all possible solutions of the given Diophantine equation.

Finding the number of solutions and the solutions in a given interval

From previous section, it should be clear that if we don't impose any restrictions on the solutions, there would be infinite number of them. So in this section, we add some restrictions on the interval of x and y , and we will try to count and enumerate all the solutions.

Let there be two intervals: $[\min_x; \max_x]$ and $[\min_y; \max_y]$ and let's say we only want to find the solutions in these two intervals.

Note that if a or b is 0 , then the problem only has one solution. We don't consider this case here.

First, we can find a solution which has minimum value of x , such that $x \geq \min_x$. To do this, we first find any solution of the Diophantine equation. Then, we shift this solution to get $x \geq \min_x$ (using what we know about the set of all solutions in previous section). This can be done in $O(1)$. Denote this minimum value of x by l_{x1} .

Similarly, we can find the maximum value of x which satisfies $x \leq \max_x$. Denote this maximum value of x by r_{x1} .

Similarly, we can find the minimum value of y ($y \geq \min_y$) and maximum value of y ($y \leq \max_y$). Denote the corresponding values of x by l_{x2} and r_{x2} .

The final solution is all solutions with x in intersection of $[l_{x1}, r_{x1}]$ and $[l_{x2}, r_{x2}]$. Let denote this intersection by $[l_x, r_x]$.

Following is the code implementing this idea. Notice that we divide a and b at the beginning by g . Since the equation $ax + by = c$ is equivalent to the equation $\frac{a}{g}x + \frac{b}{g}y = \frac{c}{g}$, we can use this one instead and have $\gcd(\frac{a}{g}, \frac{b}{g}) = 1$, which simplifies the formulas.

```
void shift_solution(int & x, int & y, int a, int b, int cnt) {
    x += cnt * b;
```

```

    y -= cnt * a;
}

int find_all_solutions(int a, int b, int c, int minx, int maxx, int
miny, int maxy) {
    int x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return 0;
    a /= g;
    b /= g;

    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx)
        shift_solution(x, y, a, b, sign_b);
    if (x > maxx)
        return 0;
    int lx1 = x;

    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx)
        shift_solution(x, y, a, b, -sign_b);
    int rx1 = x;

    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny)
        shift_solution(x, y, a, b, -sign_a);
    if (y > maxy)
        return 0;
    int lx2 = x;

    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b, sign_a);
    int rx2 = x;

    if (lx2 > rx2)
        swap(lx2, rx2);
    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);

    if (lx > rx)
        return 0;
    return (rx - lx) / abs(b) + 1;
}

```

Once we have l_x and r_x , it is also simple to enumerate through all the solutions. Just need to iterate through $x = l_x + k \cdot \frac{b}{g}$ for all $k \geq 0$ until $x = r_x$, and find the corresponding y values using the equation $ax + by = c$.

Find the solution with minimum value of $x + y$

Here, x and y also need to be given some restriction, otherwise, the answer may become negative infinity.

The idea is similar to previous section: We find any solution of the Diophantine equation, and then shift the solution to satisfy some conditions.

Finally, use the knowledge of the set of all solutions to find the minimum:

$$x' = x + k \cdot \frac{b}{g},$$

$$y' = y - k \cdot \frac{a}{g}.$$

Note that $x + y$ change as follows:

$$x' + y' = x + y + k \cdot \left(\frac{b}{g} - \frac{a}{g} \right) = x + y + k \cdot \frac{b - a}{g}$$

If $a < b$, we need to select smallest possible value of k . If $a > b$, we need to select the largest possible value of k . If $a = b$, all solution will have the same sum $x + y$.

Translated

From: e-maxx.ru

Fibonacci Numbers

The Fibonacci sequence is defined as follows:

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$$

The first elements of the sequence ([OEIS A000045](#)) are:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \dots$$

Properties

Fibonacci numbers possess a lot of interesting properties. Here are a few of them:

- Cassini's identity:

$$F_{n-1}F_{n+1} - F_n^2 = (-1)^n$$

This can be proved by induction. A one-line proof by Knuth comes from taking the determinant of the 2x2 matrix form below.

- The "addition" rule:

$$F_{n+k} = F_k F_{n+1} + F_{k-1} F_n$$

- Applying the previous identity to the case $k = n$, we get:

$$F_{2n} = F_n(F_{n+1} + F_{n-1})$$

- From this we can prove by induction that for any positive integer k , F_{nk} is multiple of F_n .
- The inverse is also true: if F_m is multiple of F_n , then m is multiple of n .
- GCD identity:

$$\text{GCD}(F_m, F_n) = F_{\text{GCD}(m,n)}$$

- Fibonacci numbers are the worst possible inputs for Euclidean algorithm (see [Lame's theorem](#) in [Euclidean algorithm](#))

Fibonacci Coding

We can use the sequence to encode positive integers into binary code words. According to Zeckendorf's theorem, any natural number n can be uniquely represented as a sum of Fibonacci numbers:

$$N = F_{k_1} + F_{k_2} + \dots + F_{k_r}$$

such that $k_1 \geq k_2 + 2$, $k_2 \geq k_3 + 2$, $\dots$, $k_r \geq 2$ (i.e.: the representation cannot use two consecutive Fibonacci numbers).

It follows that any number can be uniquely encoded in the Fibonacci coding. And we can describe this representation with binary codes $d_0 d_1 d_2 \dots d_s 1$, where d_i is 1 if F_{i+2} is used in the representation. The code will be appended by a 1 to indicate the end of the code word. Notice that this is the only occurrence where two consecutive 1-bits appear.

$$\begin{array}{llll}
 1 = 1 & = & F_2 & = & (11)_F \\
 2 = 2 & = & F_3 & = & (011)_F \\
 6 = 5 + 1 & = & F_5 + F_2 & = & (10011)_F \\
 8 = 8 & = & F_6 & = & (000011)_F \\
 9 = 8 + 1 & = & F_6 + F_2 & = & (100011)_F \\
 19 = 13 + 5 + 1 = F_7 + F_5 + F_2 & = & & = & (1001011)_F
 \end{array}$$

The encoding of an integer n can be done with a simple greedy algorithm:

1. Iterate through the Fibonacci numbers from the largest to the smallest until you find one less than or equal to n .
2. Suppose this number was F_i . Subtract F_i from n and put a 1 in the $i - 2$ position of the code word (indexing from 0 from the leftmost to the rightmost bit).
3. Repeat until there is no remainder.
4. Add a final 1 to the codeword to indicate its end.

To decode a code word, first remove the final 1. Then, if the i -th bit is set (indexing from 0 from the leftmost to the rightmost bit), sum F_{i+2} to the number.

Formulas for the n^{th} Fibonacci number

Closed-form expression

There is a formula known as "Binet's formula", even though it was already known by Moivre:

$$F_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

This formula is easy to prove by induction, but it can be deduced with the help of the concept of generating functions or by solving a functional equation.

You can immediately notice that the second term's absolute value is always less than 1, and it also decreases very rapidly (exponentially). Hence the value of the first term alone is "almost" F_n . This can be written strictly as:

$$F_n = \left\lceil \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n}{\sqrt{5}} \right\rceil$$

where the square brackets denote rounding to the nearest integer.

As these two formulas would require very high accuracy when working with fractional numbers, they are of little use in practical calculations.

Fibonacci in linear time

The n -th Fibonacci number can be easily found in $O(n)$ by computing the numbers one by one up to n . However, there are also faster ways, as we will see.

We can start from an iterative approach, to take advantage of the use of the formula $F_n = F_{n-1} + F_{n-2}$, therefore, we will simply precalculate those values in an array. Taking into account the base cases for F_0 and F_1 .

```
int fib(int n) {
    int a = 0;
    int b = 1;
    for (int i = 0; i < n; i++) {
        int tmp = a + b;
        a = b;
        b = tmp;
    }
    return a;
}
```

In this way, we obtain a linear solution, $O(n)$ time, saving all the values prior to n in the sequence.

Matrix form

To go from (F_n, F_{n-1}) to (F_{n+1}, F_n) , we can express the linear recurrence as a 2x2 matrix multiplication:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} F_n + F_{n-1} \\ F_n \end{pmatrix} = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

This lets us treat iterating the recurrence as repeated matrix multiplication, which has nice properties. In particular,

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

where $F_1 = 1, F_0 = 0$. In fact, since

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} F_2 & F_1 \\ F_1 & F_0 \end{pmatrix}$$

we can use the matrix directly:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

Thus, in order to find F_n in $O(\log n)$ time, we must raise the matrix to n . (See [Binary exponentiation](#))

```
struct matrix {
    long long mat[2][2];
    matrix friend operator *(const matrix &a, const matrix &b){
        matrix c;
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                c.mat[i][j] = 0;
                for (int k = 0; k < 2; k++) {
                    c.mat[i][j] += a.mat[i][k] * b.mat[k][j];
                }
            }
        }
        return c;
    }
};

matrix matpow(matrix base, long long n) {
    matrix ans{ {
        {1, 0},
        {0, 1}
    } };
    while (n) {
        if(n&1)
            ans = ans*base;
    }
}
```

```

        base = base*base;
        n >>= 1;
    }
    return ans;
}

long long fib(int n) {
    matrix base{ {
        {1, 1},
        {1, 0}
    } };
    return matpow(base, n).mat[0][1];
}

```

Fast Doubling Method

By expanding the above matrix expression for $n = 2 \cdot k$

$$\begin{pmatrix} F_{2k+1} & F_{2k} \\ F_{2k} & F_{2k-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{2k} = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}^2$$

we can find these simpler equations:

$$F_{2k+1} = F_{k+1}^2 + F_k^2$$

$$F_{2k} = F_k(F_{k+1} + F_{k-1}) = F_k(2F_{k+1} - F_k)$$

Thus using above two equations Fibonacci numbers can be calculated easily by the following code:

```

pair<int, int> fib (int n) {
    if (n == 0)
        return {0, 1};

    auto p = fib(n >> 1);
    int c = p.first * (2 * p.second - p.first);
    int d = p.first * p.first + p.second * p.second;
    if (n & 1)
        return {d, c + d};
    else

```

```
    return {c, d};
}
```

The above code returns F_n and F_{n+1} as a pair.

Periodicity modulo p

Consider the Fibonacci sequence modulo p . We will prove the sequence is periodic.

Let us prove this by contradiction. Consider the first $p^2 + 1$ pairs of Fibonacci numbers taken modulo p :

$$(F_0, F_1), (F_1, F_2), \dots, (F_{p^2}, F_{p^2+1})$$

There can only be p different remainders modulo p , and at most p^2 different pairs of remainders, so there are at least two identical pairs among them. This is sufficient to prove the sequence is periodic, as a Fibonacci number is only determined by its two predecessors. Hence if two pairs of consecutive numbers repeat, that would also mean the numbers after the pair will repeat in the same fashion.

We now choose two pairs of identical remainders with the smallest indices in the sequence. Let the pairs be (F_a, F_{a+1}) and (F_b, F_{b+1}) . We will prove that $a = 0$. If this was false, there would be two previous pairs (F_{a-1}, F_a) and (F_{b-1}, F_b) , which, by the property of Fibonacci numbers, would also be equal. However, this contradicts the fact that we had chosen pairs with the smallest indices, completing our proof that there is no pre-period (i.e the numbers are periodic starting from F_0).

Original

Knuth's Optimization

Knuth's optimization, also known as the Knuth-Yao Speedup, is a special case of dynamic programming on ranges, that can optimize the time complexity of solutions by a linear factor, from $O(n^3)$ for standard range DP to $O(n^2)$.

Conditions

The Speedup is applied for transitions of the form

$$dp(i, j) = \min_{i \leq k < j} [dp(i, k) + dp(k + 1, j) + C(i, j)].$$

Similar to [divide and conquer DP](#), let $opt(i, j)$ be the value of k that minimizes the expression in the transition (opt is referred to as the "optimal splitting point" further in this article). The optimization requires that the following holds:

$$opt(i, j - 1) \leq opt(i, j) \leq opt(i + 1, j).$$

We can show that it is true when the cost function C satisfies the following conditions for $a \leq b \leq c \leq d$:

1. $C(b, c) \leq C(a, d)$;
2. $C(a, c) + C(b, d) \leq C(a, d) + C(b, c)$ (the quadrangle inequality [QI]).

This result is proved further below.

Algorithm

Let's process the dp states in such a way that we calculate $dp(i, j - 1)$ and $dp(i + 1, j)$ before $dp(i, j)$, and in doing so we also calculate $opt(i, j - 1)$ and $opt(i + 1, j)$. Then for calculating $opt(i, j)$, instead of testing values of k from i to $j - 1$, we only need to test from $opt(i, j - 1)$ to $opt(i + 1, j)$. To process (i, j) pairs in this order it is sufficient to use nested for loops in which i goes from the maximum value to the minimum one and j goes from $i + 1$ to the maximum value.

Generic implementation

Though implementation varies, here's a fairly generic example. The structure of the code is almost identical to that of Range DP.

```
int solve() {
    int N;
    ... // read N and input
    int dp[N][N], opt[N][N];

    auto C = [&](int i, int j) {
        ... // Implement cost function C.
    };

    for (int i = 0; i < N; i++) {
        opt[i][i] = i;
        ... // Initialize dp[i][i] according to the problem
    }

    for (int i = N-2; i >= 0; i--) {
        for (int j = i+1; j < N; j++) {
            int mn = INT_MAX;
            int cost = C(i, j);
            for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {
                if (mn >= dp[i][k] + dp[k+1][j] + cost) {
                    opt[i][j] = k;
                    mn = dp[i][k] + dp[k+1][j] + cost;
                }
            }
            dp[i][j] = mn;
        }
    }

    return dp[0][N-1];
}
```

Complexity

A complexity of the algorithm can be estimated as the following sum:

$$\sum_{i=1}^N \sum_{j=i+1}^N [\text{opt}(i+1, j) - \text{opt}(i, j-1)] = \sum_{i=1}^N \sum_{j=i}^{N-1} [\text{opt}(i+1, j+1) - \text{opt}(i, j)]$$

As you see, most of the terms in this expression cancel each other out, except for positive terms with $j = N - 1$ and negative terms with $i = 1$. Thus, the whole sum can be estimated as

$$\sum_{k=1}^N [\text{opt}(k, N) - \text{opt}(1, k)] = O(n^2),$$

rather than $O(n^3)$ as it would be if we were using a regular range DP.

On practice

The most common application of Knuth's optimization is in Range DP, with the given transition. The only difficulty is in proving that the cost function satisfies the given conditions. The simplest case is when the cost function $C(i, j)$ is simply the sum of the elements of the subarray $S[i, i+1, \dots, j]$ for some array (depending on the question). However, they can be more complicated at times.

Note that more than the conditions on the dp transition and the cost function, the key to this optimization is the inequality on the optimum splitting point. In some problems, such as the optimal binary search tree problem (which is, incidentally, the original problem for which this optimization was developed), the transitions and cost functions will be less obvious, however, one can still prove that $\text{opt}(i, j-1) \leq \text{opt}(i, j) \leq \text{opt}(i+1, j)$, and thus, use this optimization.

Proof of correctness

To prove the correctness of this algorithm in terms of $C(i, j)$ conditions, it suffices to prove that

$$\text{opt}(i, j - 1) \leq \text{opt}(i, j) \leq \text{opt}(i + 1, j)$$

assuming the given conditions are satisfied.

 Lemma

$\text{dp}(i, j)$ also satisfies the quadrangle inequality, given the conditions of the problem are satisfied.

 Proof



The proof for this lemma uses strong induction. It has been taken from the paper [Efficient Dynamic Programming Using Quadrangle Inequalities](#), authored by F. Frances Yao, which introduced the Knuth-Yao Speedup (this particular statement is Lemma 2.1 in the paper). The idea is to induct on the length $l = d - a$. The case where $l = 1$ is trivial. For $l > 1$ consider 2 cases:

1. $b = c$

The inequality reduces to $dp(a, b) + dp(b, d) \leq dp(a, d)$ (This assumes that $dp(i, i) = 0$ for all i , which is the case for all problems using this optimization). Let $opt(a, d) = z$.

- If $z < j$,
Note that

$$dp(a, b) \leq dp_z(a, b) = dp(a, z) + dp(z + 1, b) + C(a, b).$$

Therefore,

$$dp(a, b) + dp(b, d) \leq dp(a, z) + dp(z + 1, b) + dp(b, d) + C(a, b)$$

From the induction hypothesis, $dp(z + 1, b) + dp(b, d) \leq dp(z + 1, d)$. Also, it is given that $C(a, b) \leq C(a, d)$. Combining these 2 facts with above inequality yields the desired result.

- If $z \geq j$, the proof of this case is symmetric to the previous case.

2. $b < c$

Let $opt(b, c) = z$ and $opt(a, d) = y$.

- If $z \leq y$,

$$dp(a, c) + dp(b, d) \leq dp_z(a, c) + dp_y(b, d)$$

where

$$dp_z(a, c) + dp_y(b, d) = C(a, c) + C(b, d) + dp(a, z) + dp(z + 1, c) + \dots$$



Using the QI on C and on the dp state for the indices $z + 1 \leq y + 1 \leq c \leq d$ (from the induction hypothesis) yields the desired result.

- If $z > y$, the proof of this case is symmetric to the previous case.

This completes the proof of the lemma.

Now, consider the following setup. We have 2 indices $i \leq p \leq q < j$. Set $dp_k = C(i, j) + dp(i, k) + dp(k + 1, j)$.

Suppose we show that

$$dp_p(i, j - 1) \geq dp_q(i, j - 1) \implies dp_p(i, j) \geq dp_q(i, j).$$

Setting $q = \text{opt}(i, j - 1)$, by definition, $dp_p(i, j - 1) \geq dp_q(i, j - 1)$. Therefore, applying the inequality to all $i \leq p \leq q$, we can infer that $\text{opt}(i, j)$ is at least as much as $\text{opt}(i, j - 1)$, proving the first half of the inequality.

Now, using the QI on some indices $p + 1 \leq q + 1 \leq j - 1 \leq j$, we get

$$\begin{aligned} & dp(p + 1, j - 1) + dp(q + 1, j) \leq dp(q + 1, j - 1) + dp(p + 1, j) \\ \implies & (dp(i, p) + dp(p + 1, j - 1) + C(i, j - 1)) + (dp(i, q) + dp(q + 1, j) + C(q + 1, j)) \\ & \leq (dp(i, q) + dp(q + 1, j - 1) + C(i, j - 1)) + (dp(i, p) + dp(p + 1, j) + C(p + 1, j)) \\ \implies & dp_p(i, j - 1) + dp_q(i, j) \leq dp_p(i, j) + dp_q(i, j - 1) \\ \implies & dp_p(i, j - 1) - dp_q(i, j - 1) \leq dp_p(i, j) - dp_q(i, j) \end{aligned}$$

Finally,

$$\begin{aligned} & dp_p(i, j - 1) \geq dp_q(i, j - 1) \\ \implies & 0 \leq dp_p(i, j - 1) - dp_q(i, j - 1) \leq dp_p(i, j) - dp_q(i, j) \\ \implies & dp_p(i, j) \geq dp_q(i, j) \end{aligned}$$

This proves the first part of the inequality, i.e., $\text{opt}(i, j - 1) \leq \text{opt}(i, j)$. The second part $\text{opt}(i, j) \leq \text{opt}(i + 1, j)$ can be shown with the same idea, starting with the

inequality $dp(i, p) + dp(i + 1, q) \leq dp(i + 1, p) + dp(i, q)$.

This completes the proof.

References

- [Geeksforgeeks Article](#)
- [Doc on DP Speedups](#)
- [Efficient Dynamic Programming Using Quadrangle Inequalities](#)

Contributors:

[Oleksandr Kulkov](#) (70.77%) [DrDrunkenstein42](#) (28.21%) [FloppaInspector](#) (0.51%)
[Michael Hayter](#) (0.51%)

Translated

From: e-maxx.ru

Finding a negative cycle in the graph

You are given a directed weighted graph G with N vertices and M edges. Find any cycle of negative weight in it, if such a cycle exists.

In another formulation of the problem you have to find all pairs of vertices which have a path of arbitrarily small weight between them.

It is convenient to use different algorithms to solve these two variations of the problem, so we'll discuss both of them here.

Using Bellman-Ford algorithm

Bellman-Ford algorithm allows you to check whether there exists a cycle of negative weight in the graph, and if it does, find one of these cycles.

The details of the algorithm are described in the article on the [Bellman-Ford](#) algorithm. Here we'll describe only its application to this problem.

The standard implementation of Bellman-Ford looks for a negative cycle reachable from some starting vertex v ; however, the algorithm can be modified to just looking for any negative cycle in the graph. For this we need to put all the distance $d[i]$ to zero and not infinity – as if we are looking for the shortest path from all vertices simultaneously; the validity of the detection of a negative cycle is not affected.

Do N iterations of Bellman-Ford algorithm. If there were no changes on the last iteration, there is no cycle of negative weight in the graph. Otherwise take a vertex the distance to which has changed, and go from it via its ancestors until a cycle is found. This cycle will be the desired cycle of negative weight.

Implementation

```

struct Edge {
    int a, b, cost;
};

int n;
vector<Edge> edges;
const int INF = 1000000000;

void solve() {
    vector<int> d(n, 0);
    vector<int> p(n, -1);
    int x;

    for (int i = 0; i < n; ++i) {
        x = -1;
        for (Edge e : edges) {
            if (d[e.a] + e.cost < d[e.b]) {
                d[e.b] = max(-INF, d[e.a] + e.cost);
                p[e.b] = e.a;
                x = e.b;
            }
        }
    }

    if (x == -1) {
        cout << "No negative cycle found.";
    } else {
        for (int i = 0; i < n; ++i)
            x = p[x];

        vector<int> cycle;
        for (int v = x; v = p[v]) {
            cycle.push_back(v);
            if (v == x && cycle.size() > 1)
                break;
        }
        reverse(cycle.begin(), cycle.end());

        cout << "Negative cycle: ";
        for (int v : cycle)
            cout << v << ' ';
        cout << endl;
    }
}

```

Using Floyd-Warshall algorithm

The Floyd-Warshall algorithm allows to solve the second variation of the problem - finding all pairs of vertices (i, j) which don't have a shortest path between them (i.e. a path of arbitrarily small weight exists).

Again, the details can be found in the [Floyd-Warshall](#) article, and here we describe only its application.

Run Floyd-Warshall algorithm on the graph. Initially $d[v][v] = 0$ for each v . But after running the algorithm $d[v][v]$ will be smaller than 0 if there exists a negative length path from v to v . We can use this to also find all pairs of vertices that don't have a shortest path between them. We iterate over all pairs of vertices (i, j) and for each pair we check whether they have a shortest path between them. To do this try all possibilities for an intermediate vertex t . (i, j) doesn't have a shortest path, if one of the intermediate vertices t has $d[t][t] < 0$ (i.e. t is part of a cycle of negative weight), t can be reached from i and j can be reached from t . Then the path from i to j can have arbitrarily small weight. We will denote this with `-INF`.

Implementation

```
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        for (int t = 0; t < n; ++t) {
            if (d[i][t] < INF && d[t][t] < 0 && d[t][j] < INF)
                d[i][j] = - INF;
        }
    }
}
```

Minimum Enclosing Circle

Consider the following problem:

Library Checker - Minimum Enclosing Circle

You're given $n \leq 10^5$ points $p_i = (x_i, y_i)$.

For each p_i , find whether it lies on the circumference of the minimum enclosing circle of $\{p_1, \dots, p_n\}$.

Here, by the minimum enclosing circle (MEC) we mean a circle with minimum possible radius that contains all the n points, inside the circle or on its boundary. This problem has a simple randomized solution that, on first glance, looks like it would run in $O(n^3)$, but actually works in $O(n)$ expected time.

To better understand the reasoning below, we should immediately note that the solution to the problem is unique:

Why is the MEC unique?

Consider the following setup: Let r be the radius of the MEC. We draw a circle of radius r around each of the points $p_1, \dots, p_n$. Geometrically, the centers of circles that have radius r and cover all the points $p_1, \dots, p_n$ form the intersection of all n circles.

Now, if the intersection is just a single point, this already proves that it is unique. Otherwise, the intersection is a shape of non-zero area, so we can reduce r by a tiny bit, and still have non-empty intersection, which contradicts the assumption that r was the minimum possible radius of the enclosing circle.

With a similar logic, we can also show the uniqueness of the MEC if we additionally demand that it passes through a given specific point p_i or two points p_i and p_j (it is also unique because its radius uniquely defines it).

Alternatively, we can also assume that there are two MECs, and then notice that their intersection (which contains the points $p_1, \dots, p_n$ already) must have a smaller diameter than initial circles, and thus can be covered with a smaller circle.

Welzl's algorithm

For brevity, let's denote $\text{mec}(p_1, \dots, p_n)$ to be the MEC of $\{p_1, \dots, p_n\}$, and let $P_i = \{p_1, \dots, p_i\}$.

The algorithm, initially [proposed](#) by Welzl in 1991, goes as follows:

1. Apply a random permutation to the input sequence of points.
2. Maintain the current candidate to be the MEC C , starting with $C = \text{mec}(p_1, p_2)$.
3. Iterate over $i = 3..n$ and check if $p_i \in C$.

- a. If $p_i \in C$ it means that C is the MEC of P_i .
- b. Otherwise, assign $C = \text{mec}(p_i, p_1)$ and iterate over $j = 2..i$ and check if $p_j \in C$.
 - i. If $p_j \in C$, then C is the MEC of P_j among circles that pass through p_i .
 - ii. Otherwise, assign $C = \text{mec}(p_i, p_j)$ and iterate over $k = 1..j$ and check if $p_k \in C$.
 - A. If $p_k \in C$, then C is the MEC of P_k among circles that pass through p_i and p_j .
 - B. Otherwise, $C = \text{mec}(p_i, p_j, p_k)$ is the MEC of P_k among circles that pass through p_i and p_j .

We can see that each level of nestedness here has an invariant to maintain (that C is the MEC among circles that also pass through additionally given 0, 1 or 2 points), and whenever the inner loop closes, its invariant becomes equivalent to the invariant of the current iteration of its parent loop. This, in turn, ensures the *correctness* of the algorithm as a whole.

Omitting some technical details, for now, the whole algorithm can be implemented in C++ as follows:

```
struct point {...};

// Is represented by 2 or 3 points on its circumference
struct mec {...};

bool inside(mec const& C, point p) {
    return ...;
}

// Choose some good generator of randomness for the shuffle
mt19937_64 gen(...);
mec enclosing_circle(vector<point> &p) {
    ranges::shuffle(p, gen);
    auto C = mec{p[0], p[1]};
    for(int i = 0; i < n; i++) {
        if(!inside(C, p[i])) {
            C = mec{p[i], p[0]};
            for(int j = 0; j < i; j++) {
                if(!inside(C, p[j])) {
                    C = mec{p[i], p[j]};
                    for(int k = 0; k < j; k++) {
                        if(!inside(C, p[k])) {
                            C = mec{p[i], p[j], p[k]};
                        }
                    }
                }
            }
        }
    }
    return C;
}
```

Now, it is to be expected that checking that a point p_i is inside the MEC of 2 or 3 points can be done in $O(1)$ (we will discuss this later on). But even then, the algorithm above looks as if it would take $O(n^3)$ in the worst case just because of all the nested loops. So, how come we claimed the linear expected runtime? Let's figure out!

Complexity analysis

For the inner-most loop (over k), clearly its expected runtime is $O(j)$ operations. What about the loop over j ?

It only triggers the next loop if p_j is on the boundary of the MEC of P_j that also passes through point i , and removing p_j would further shrink the circle. Of all points in P_j there can only be at most 2 points with such property, because if there are more than 2 points from P_j on the boundary, it means that after removing any of them, there will still be at least 3 points on the boundary, sufficient to uniquely define the circle.

In other words, after initial random shuffle, there is at most $\frac{2}{j}$ probability that we get one of the at most two unlucky points as p_j . Summing it up over all j from 1 to i , we get the expected runtime of

$$\sum_{j=1}^i \frac{2}{j} \cdot O(j) = O(i).$$

In exactly same fashion we can now also prove that the outermost loop has expected runtime of $O(n)$.

Checking that a point is in the MEC of 2 or 3 points

Let's now figure out the implementation detail of `point` and `mec`. In this problem, it turns out to be particularly useful to use `std::complex` as a class for points:

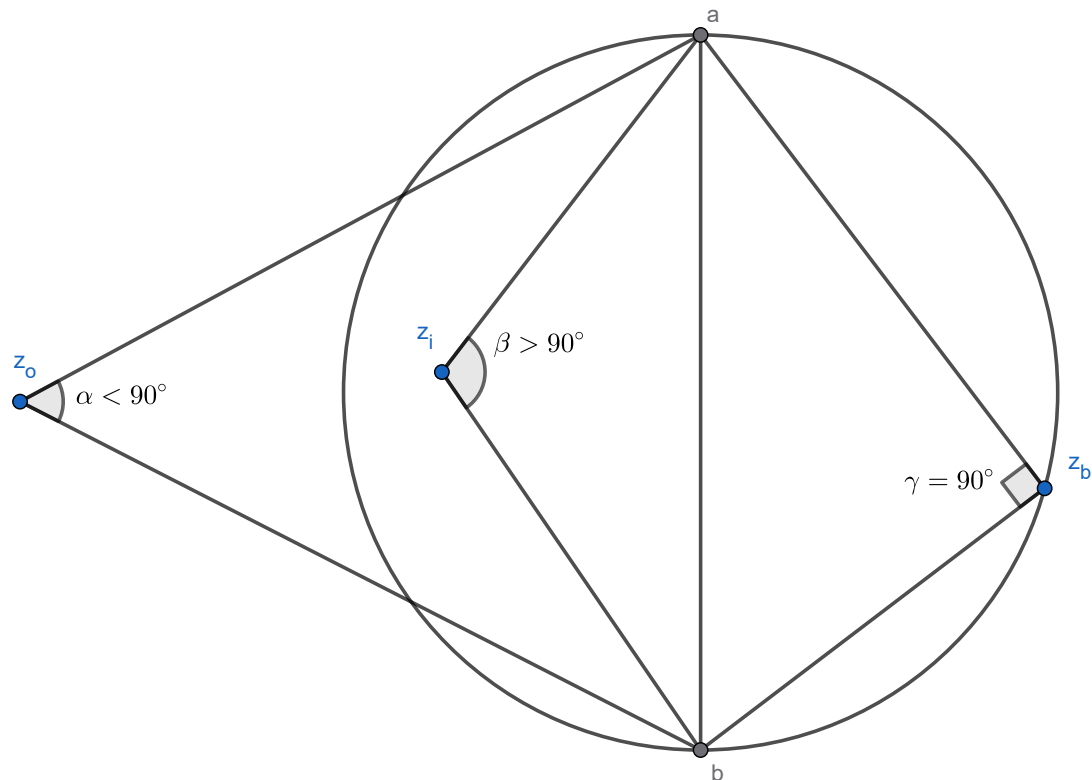
```
using ftype = int64_t;
using point = complex<ftype>;
```

As a reminder, a complex number is a number of type $x + yi$, where $i^2 = -1$ and $x, y \in \mathbb{R}$. In C++, such complex number is represented by a 2-dimensional point (x, y) . Complex numbers already implement basic component-wise linear operations (addition, multiplication by a real number), but also their multiplication and division carry certain geometric meaning.

Without going in too much detail, we will note the most important property for this particular task: Multiplying two complex numbers adds up their polar angles (counted from Ox counter-clockwise), and taking a conjugate (i.e. changing $z = x + yi$ into $\bar{z} = x - yi$) multiplies the polar angle with -1 . This allows us to formulate some very simple criteria for whether a point z is inside the MEC of 2 or 3 specific points.

MEC of 2 points

For 2 points a and b , their MEC is simply the circle centered at $\frac{a+b}{2}$ with the radius $\frac{|a-b|}{2}$, in other words the circle that has ab as a diameter. To check if z is inside this circle we simply need to check that the angle between za and zb is not acute.



Inner angles are obtuse, external angles are acute and angles on the circumference are right

Equivalently, we need to check that

$$I_0 = (b - z)\overline{(a - z)}$$

doesn't have a positive real coordinate (corresponding to points that have a polar angle between -90° and 90°).

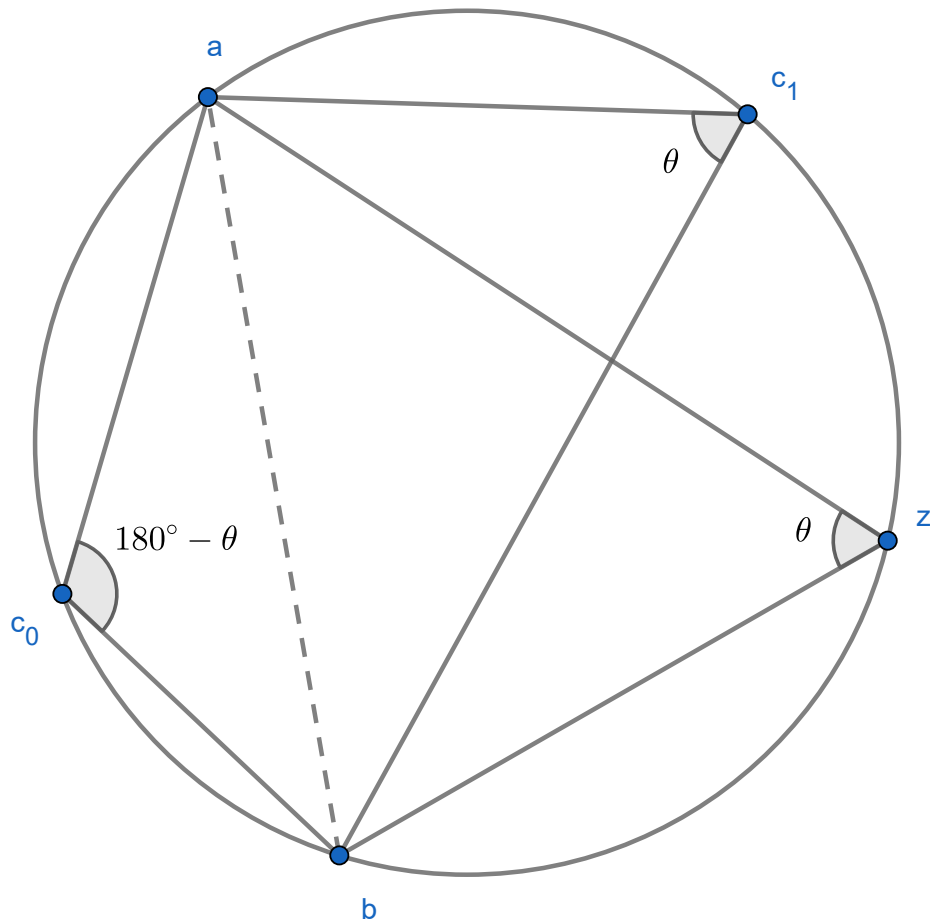
MEC of 3 points

Adding z to the triangle abc will make it a quadrilateral. Consider the following expression:

$$\angle azb + \angle bca$$

In a **cyclic quadrilateral**, if c and z are from the same side of ab , then the angles are equal, and will add up to 0° when summed up signed (i.e. positive if counter-clockwise and negative if clockwise).

Correspondingly, if c and z are on the opposite sides, the angles will add up to 180° .



Adjacent inscribed angles are same, opposing angles complement to 180 degrees

In terms of complex numbers, we can note that $\angle azb$ is the polar angle of $(b - z)\overline{(a - z)}$ and $\angle bca$ is the polar angle of $(a - c)\overline{(b - c)}$. Thus, we can conclude that $\angle azb + \angle bca$ is the polar angle of

$$I_1 = (b - z)\overline{(a - z)}(a - c)\overline{(b - c)}$$

If the angle is 0° or 180° , it means that the imaginary part of I_1 is 0, otherwise we can deduce whether z is inside or outside of the enclosing circle of abc by checking the sign of the imaginary part of I_1 . Positive imaginary part corresponds to positive angles, and negative imaginary part corresponds to negative angles.

But which one of them means that z is inside or outside of the circle? As we already noticed, having z inside the circle generally increases the magnitude of $\angle azb$, while having it outside the circle decreases it. As such, we have the following 4 cases:

1. $\angle bca > 0^\circ$, c on the same side of ab as z . Then, $\angle azb < 0^\circ$, and $\angle azb + \angle bca < 0^\circ$ for points inside the circle.
2. $\angle bca < 0^\circ$, c on the same side of ab as z . Then, $\angle azb > 0^\circ$, and $\angle azb + \angle bca > 0^\circ$ for points inside the circle.

3. $\angle bca > 0^\circ$, c on the opposite side of ab to z . Then, $\angle azb > 0^\circ$ and $\angle azb + \angle bca > 180^\circ$ for points inside the circle.
4. $\angle bca < 0^\circ$, c on the opposite side of ab to z . Then, $\angle azb < 0^\circ$ and $\angle azb + \angle bca < 180^\circ$ for points inside the circle.

In other words, if $\angle bca$ is positive, points inside the circle will have $\angle azb + \angle bca < 0^\circ$, otherwise they will have $\angle azb + \angle bca > 0^\circ$, assuming that we normalize the angles between -180° and 180° . This, in turn, can be checked by the signs of imaginary parts of $I_2 = (a - c)\overline{(b - c)}$ and $I_1 = I_0 I_2$.

Note: As we multiply four complex numbers to get I_1 , the intermediate coefficients can be as large as $O(A^4)$, where A is the largest coordinate magnitude in the input. On the bright side, if the input is integer, both checks above can be done fully in integers.

Implementation

Now, to actually implement the check, we should first decide how to represent the MEC. As our criteria work with the points directly, a natural and efficient way to do this is to say that MEC is directly represented as a pair or triple of points that defines it:

```
using mec = variant<
    array<point, 2>,
    array<point, 3>
>;
```

Now, we can use `std::visit` to efficiently deal with both cases in accordance with criteria above:

```
/* I < 0 if z inside C,
   I > 0 if z outside C,
   I = 0 if z on the circumference of C */
ftype indicator(mec const& C, point z) {
    return visit([&](auto &&C) {
        point a = C[0], b = C[1];
        point I0 = (b - z) * conj(a - z);
        if constexpr (size(C) == 2) {
            return real(I0);
        } else {
            point c = C[2];
            point I2 = (a - c) * conj(b - c);
            point I1 = I0 * I2;
            return imag(I2) < 0 ? -imag(I1) : imag(I1);
        }
    }, C);
}

bool inside(mec const& C, point p) {
    return indicator(C, p) <= 0;
}
```

Now, we can finally ensure that everything works by submitting the problem to the Library Checker: [#308668](#).

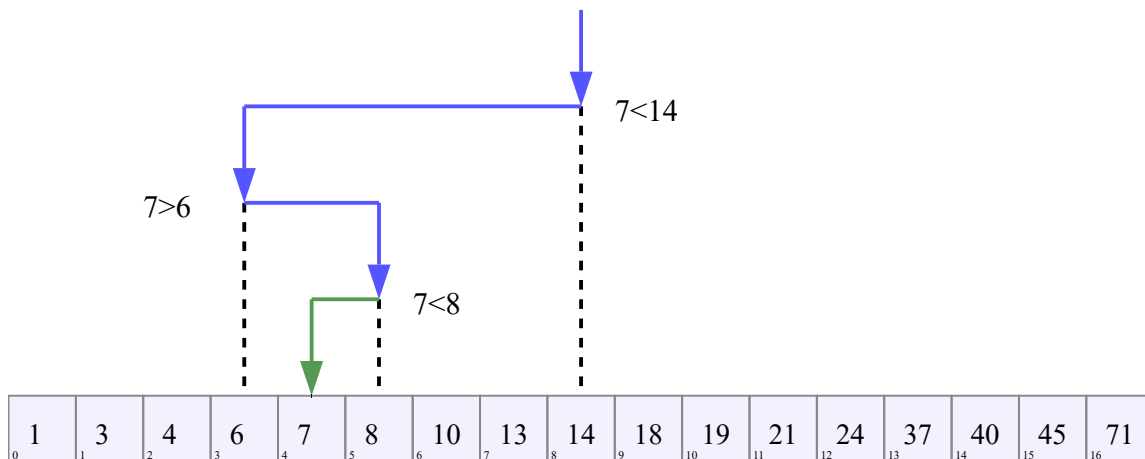
Original

Binary search

Binary search is a method that allows for quicker search of something by splitting the search interval into two. Its most common application is searching values in sorted arrays, however the splitting idea is crucial in many other typical tasks.

Search in sorted arrays

The most typical problem that leads to the binary search is as follows. You're given a sorted array $A_0 \leq A_1 \leq \dots \leq A_{n-1}$, check if k is present within the sequence. The simplest solution would be to check every element one by one and compare it with k (a so-called linear search). This approach works in $O(n)$, but doesn't utilize the fact that the array is sorted.



Binary search of the value 7 in an array.

The image by AlwaysAngry is distributed under CC BY-SA 4.0 license.

Now assume that we know two indices $L < R$ such that $A_L \leq k \leq A_R$. Because the array is sorted, we can deduce that k either occurs among $A_L, A_{L+1}, \dots, A_R$ or doesn't occur in the array at all. If we pick an arbitrary index M such that $L < M < R$ and check whether k is less or greater than A_M . We have two possible cases:

1. $A_L \leq k \leq A_M$. In this case, we reduce the problem from $[L, R]$ to $[L, M]$;
2. $A_M \leq k \leq A_R$. In this case, we reduce the problem from $[L, R]$ to $[M, R]$.

When it is impossible to pick M , that is, when $R = L + 1$, we directly compare k with A_L and A_R . Otherwise we would want to pick M in such manner that it reduces the active segment to a single element as quickly as possible in the worst case.

Since in the worst case we will always reduce to larger segment of $[L, M]$ and $[M, R]$. Thus, in the worst case scenario the reduction would be from $R - L$ to $\max(M - L, R - M)$. To minimize this value, we should pick $M \approx \frac{L+R}{2}$, then

$$M - L \approx \frac{R - L}{2} \approx R - M.$$

In other words, from the worst-case scenario perspective it is optimal to always pick M in the middle of $[L, R]$ and split it in half. Thus, the active segment halves on each step until it becomes of size 1. So, if the process needs h steps, in the end it reduces the difference between R and L from $R - L$ to $\frac{R-L}{2^h} \approx 1$, giving us the equation $2^h \approx R - L$.

Taking $\log_2$ on both sides, we get $h \approx \log_2(R - L) \in O(\log n)$.

Logarithmic number of steps is drastically better than that of linear search. For example, for $n \approx 2^{20} \approx 10^6$ you'd need to make approximately a million operations for linear search, but only around 20 operations with the binary search.

Lower bound and upper bound

It is often convenient to find the position of the first element that is greater or equal than k (called the lower bound of k in the array) or the position of the first element that is greater than k (called the upper bound of k) rather than the exact position of the element.

Together, lower and upper bounds produce a possibly empty half-interval of the array elements that are equal to k . To check whether k is present in the array it's enough to find its lower bound and check if the corresponding element equates to k .

Implementation

The explanation above provides a rough description of the algorithm. For the implementation details, we'd need to be more precise.

We will maintain a pair $L < R$ such that $A_L \leq k < A_R$. Meaning that the active search interval is $[L, R)$. We use half-interval here instead of a segment $[L, R]$ as it turns out to require less corner case work.

When $R = L + 1$, we can deduce from definitions above that R is the upper bound of k . It is convenient to initialize R with past-the-end index, that is $R = n$ and L with before-the-beginning index, that is $L = -1$. It is fine as long as we never evaluate A_L and A_R in our algorithm directly, formally treating it as $A_L = -\infty$ and $A_R = +\infty$.

Finally, to be specific about the value of M we pick, we will stick with $M = \lfloor \frac{L+R}{2} \rfloor$.

Then the implementation could look like this:

```
... // a sorted array is stored as a[0], a[1], ..., a[n-1]
int l = -1, r = n;
while (r - l > 1) {
    int m = (l + r) / 2;
    if (k < a[m]) {
        r = m; // a[l] <= k < a[m] <= a[r]
    } else {
        l = m; // a[l] <= a[m] <= k < a[r]
    }
}
```

During the execution of the algorithm, we never evaluate neither A_L nor A_R , as $L < M < R$. In the end, L will be the index of the last element that is not greater than k (or -1 if there is no such element) and R will be

the index of the first element larger than k (or n if there is no such element).

Note. Calculating m as $m = (r + 1) / 2$ can lead to overflow if l and r are two positive integers, and this error lived about 9 years in JDK as described in the [blogpost](#). Some alternative approaches include e.g. writing $m = 1 + (r - 1) / 2$ which always works for positive integer l and r , but might still overflow if l is a negative number. If you use C++20, it offers an alternative solution in the form of `m = std::midpoint(l, r)` which always works correctly.

Search on arbitrary predicate

Let $f : \{0, 1, \dots, n-1\} \rightarrow \{0, 1\}$ be a boolean function defined on $0, 1, \dots, n-1$ such that it is monotonously increasing, that is

$$f(0) \leq f(1) \leq \dots \leq f(n-1).$$

The binary search, the way it is described above, finds the partition of the array by the predicate $f(M)$, holding the boolean value of $k < A_M$ expression. It is possible to use arbitrary monotonous predicate instead of $k < A_M$. It is particularly useful when the computation of $f(k)$ requires too much time to actually compute it for every possible value. In other words, binary search finds the unique index L such that $f(L) = 0$ and $f(R) = f(L+1) = 1$ if such a *transition point* exists, or gives us $L = n-1$ if $f(0) = \dots = f(n-1) = 0$ or $L = -1$ if $f(0) = \dots = f(n-1) = 1$.

Proof of correctness supposing a transition point exists, that is $f(0) = 0$ and $f(n-1) = 1$: The implementation maintains the *loop invariant* $f(l) = 0, f(r) = 1$. When $r - l > 1$, the choice of m means $r - l$ will always decrease. The loop terminates when $r - l = 1$, giving us our desired transition point.

```
... // f(i) is a boolean function such that f(0) <= ... <= f(n-1)
int l = -1, r = n;
while (r - l > 1) {
    int m = (l + r) / 2;
    if (f(m)) {
        r = m; // 0 = f(l) < f(m) = 1
    } else {
        l = m; // 0 = f(m) < f(r) = 1
    }
}
```

Binary search on the answer

Such situation often occurs when we're asked to compute some value, but we're only capable of checking whether this value is at least i . For example, you're given an array $a_1, \dots, a_n$ and you're asked to find the maximum floored average sum

$$\left\lfloor \frac{a_l + a_{l+1} + \dots + a_r}{r - l + 1} \right\rfloor$$

among all possible pairs of l, r such that $r - l \geq x$. One of simple ways to solve this problem is to check whether the answer is at least λ , that is if there is a pair l, r such that the following is true:

$$\frac{a_l + a_{l+1} + \dots + a_r}{r - l + 1} \geq \lambda.$$

Equivalently, it rewrites as

$$(a_l - \lambda) + (a_{l+1} - \lambda) + \dots + (a_r - \lambda) \geq 0,$$

so now we need to check whether there is a subarray of a new array $a_i - \lambda$ of length at least $x + 1$ with non-negative sum, which is doable with some prefix sums.

Continuous search

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a real-valued function that is continuous on a segment $[L, R]$.

Without loss of generality assume that $f(L) \leq f(R)$. From [intermediate value theorem](#) it follows that for any $y \in [f(L), f(R)]$ there is $x \in [L, R]$ such that $f(x) = y$. Note that, unlike previous paragraphs, the function is *not* required to be monotonous.

The value x could be approximated up to $\pm\delta$ in $O\left(\log \frac{R-L}{\delta}\right)$ time for any specific value of δ . The idea is essentially the same, if we take $M \in (L, R)$ then we would be able to reduce the search interval to either $[L, M]$ or $[M, R]$ depending on whether $f(M)$ is larger than y . One common example here would be finding roots of odd-degree polynomials.

For example, let $f(x) = x^3 + ax^2 + bx + c$. Then $f(L) \rightarrow -\infty$ and $f(R) \rightarrow +\infty$ with $L \rightarrow -\infty$ and $R \rightarrow +\infty$. Which means that it is always possible to find sufficiently small L and sufficiently large R such that $f(L) < 0$ and $f(R) > 0$. Then, it is possible to find with binary search arbitrarily small interval containing x such that $f(x) = 0$.

Search with powers of 2

Another noteworthy way to do binary search is, instead of maintaining an active segment, to maintain the current pointer i and the current power k . The pointer starts at $i = L$ and then on each iteration one tests the predicate at point $i + 2^k$. If the predicate is still 0, the pointer is advanced from i to $i + 2^k$, otherwise it stays the same, then the power k is decreased by 1.

This paradigm is widely used in tasks around trees, such as finding lowest common ancestor of two vertices or finding an ancestor of a specific vertex that has a certain height. It could also be adapted to e.g. find the k -th non-zero element in a Fenwick tree.

Parallel Binary Search

- Note: Section Added my me.

When we are faced with multiple queries that can each be solved with a binary search, it is sometimes too slow to solve them one by one. Parallel Binary Search is a technique that allows us to solve all of these queries simultaneously, often leading to a significant performance improvement. The main idea is to perform the binary search for all queries at the same time (in parallel), step by step. This is particularly effective when the check function for the binary search is costly and can be optimized by processing queries in batches.

Consider a scenario where we have Q queries, and for each query q , we need to find the smallest value x that satisfies a certain condition $P(q, x)$. If $P(q, x)$ is monotonic on x , we can use binary search for each query. This would result in a total complexity of $O(Q \cdot \log(\text{range}) \cdot T_{\text{check}})$, where T_{check} is the time to evaluate $P(q, x)$.

Parallel binary search optimizes this by changing the order of operations. Instead of processing each query independently, we process all queries simultaneously, step by step. In each step of the binary search, we compute the middle points m_i for all queries and group the queries by their middle point. This is particularly powerful if the check function $P(q, x)$ has a structure that allows for efficient batching or updates.

Specifically, the major performance gain comes from two scenarios: 1. **Batching expensive checks:** If multiple queries need to check the same value m in a given step, we can perform the expensive part of the check only once and reuse the result. 2. **Efficiently updatable checks:** Often, the check for a value m (e.g., "process the first m events") can be performed much faster if we have already computed the state for $m - 1$. By processing the check values m in increasing order, we can update the state from one check to the next, instead of recomputing from scratch each time. This is a very common pattern in problems involving time or a sequence of updates.

This "offline" processing of queries, where we collect all queries and answer them together in a way that is convenient for our data structures, is the core idea behind parallel binary search.

Implementation

Imagine that we want to answer Z queries about the index of the largest value less than or equal to some X_i (for $i = 1, 2, \dots, Z$) in a sorted 0-indexed array A . Naturally, each query can be answered using binary search.

Specifically, let us consider the following array $A = [1, 3, 5, 7, 9, 9, 13, 15]$ with queries: $X = [8, 11, 4, 5]$. We can use binary search for each query sequentially.

Query	$X_1 = 8$	$X_2 = 11$	$X_3 = 4$	$X_4 = 5$
Step 1	Answer in $[0, 8)$ Check A_4 $X_1 < A_4 = 9$	Answer in $[0, 8)$ Check A_4 $X_2 \geq A_4 = 9$	Answer in $[0, 8)$ Check A_4 $X_3 < A_4 = 9$	Answer in $[0, 8)$ Check A_4 $X_4 < A_4 = 9$
Step 2	Answer in $[0, 4)$ Check A_2 $X_1 \geq A_2 = 5$	Answer in $[4, 8)$ Check A_6 $X_2 < A_6 = 13$	Answer in $[0, 4)$ Check A_2 $X_3 < A_2 = 5$	Answer in $[0, 4)$ Check A_2 $X_4 \geq A_2 = 5$
Step 3	Answer in $[2, 4)$ Check A_3 $X_1 \geq A_3 = 7$	Answer in $[4, 6)$ Check A_5 $X_2 \geq A_5 = 9$	Answer in $[0, 2)$ Check A_1 $X_3 \geq A_1 = 3$	Answer in $[2, 4)$ Check A_3 $X_4 < A_3 = 7$
Step 4	Answer in $[3, 4)$ $index = 3$	Answer in $[5, 6)$ $index = 5$	Answer in $[1, 2)$ $index = 1$	Answer in $[2, 3)$ $index = 2$

We generally process this table by columns (queries), but notice that in each row we often repeat access to certain values of the array. To limit access to these values, we can process the table by rows (steps). This does not make huge difference in our small example problem (as we can access all elements in $O(1)$), but in more complex problems, where computing these values is more complicated, this might be essential to solve these problems efficiently. Moreover, note that we can arbitrarily choose the order in which we answer questions in a single row. Let us look at the code implementing this approach.

```

// Computes the index of the largest value in a sorted array A less than or equal to X_i for
// all i.
vector<int> parallel_binary_search(vector<int>& A, vector<int>& X) {
    int N = A.size();
    int Z = X.size();
    vector<int> l(Z, -1), r(Z, N);

    for (int step = 1; step <= ceil(log2(N)); ++step) {
        // A vector of vectors to store indices of queries for each middle point.
        vector<vector<int>> m_to_queries(N);

        // Group queries by their middle point.
        for (int i = 0; i < Z; ++i) {
            if (l[i] < r[i] - 1) {
                int m = l[i] + (r[i] - l[i]) / 2;
                m_to_queries[m].push_back(i);
            }
        }

        // Process each group of queries.
        for (int m = 0; m < N; ++m) {
            if (m_to_queries[m].empty()) {
                continue;
            }
            for (int query : m_to_queries[m]) {
                if (X[query] < A[m]) {
                    r[query] = m;
                } else {
                    l[query] = m;
                }
            }
        }
    }
    return l;
}

```

Example Problem: Meteors

A pretty well known problem using this method is called "Meteors", and is listed in the practice problems. We are given N countries, and for each country, a target number of meteors to collect. We are also given a sequence of K meteor showers, each affecting a range of countries. The goal is to find, for each country, the earliest time (i.e., which meteor shower) they reach their target.

For a single country, we could binary search for the answer from 1 to K . The check for a given time t would involve summing up the meteors from the first t showers for that country. A naive check takes $O(t)$ time, leading to an overall complexity of $O(K \log K)$ for one country, and $O(N \cdot K \log K)$ for all, which is too slow.

With parallel binary search, we search for the answer for all N countries at once. In each of the $O(\log K)$ steps, we have a set of check values t_i for the countries. We can process these t_i in increasing order. To perform the check for time t , we can use a data structure like a Fenwick tree or a segment tree to maintain the meteor counts for all countries. When moving from checking time t_i to t_{i+1} , we only need to add the effects of showers from $t_i + 1$ to t_{i+1} to our data structure. This "update" approach is much faster than recomputing from scratch. The total complexity becomes $O((N + K) \log N \log K)$.

Translated

From: e-maxx.ru

Ternary Search

We are given a function $f(x)$ which is unimodal on an interval $[l, r]$. By unimodal function, we mean one of two behaviors of the function:

1. The function strictly increases first, reaches a maximum (at a single point or over an interval), and then strictly decreases.
2. The function strictly decreases first, reaches a minimum, and then strictly increases.

In this article, we will assume the first scenario. The second scenario is completely symmetrical to the first.

The task is to find the maximum of function $f(x)$ on the interval $[l, r]$.

Algorithm

Consider any 2 points m_1 , and m_2 in this interval: $l < m_1 < m_2 < r$. We evaluate the function at m_1 and m_2 , i.e. find the values of $f(m_1)$ and $f(m_2)$. Now, we get one of three options:

- $f(m_1) < f(m_2)$

The desired maximum can not be located on the left side of m_1 , i.e. on the interval $[l, m_1]$, since either both points m_1 and m_2 or just m_1 belong to the area where the function increases. In either case, this means that we have to search for the maximum in the segment $[m_1, r]$.

- $f(m_1) > f(m_2)$

This situation is symmetrical to the previous one: the maximum can not be located on the right side of m_2 , i.e. on the interval $[m_2, r]$, and the search space is reduced to the segment $[l, m_2]$.

- $f(m_1) = f(m_2)$

We can see that either both of these points belong to the area where the value of the function is maximized, or m_1 is in the area of increasing values and m_2 is in the area of descending values (here we used the strictness of function increasing/decreasing). Thus, the search space is reduced to $[m_1, m_2]$. To simplify the code, this case can be combined with any of the previous cases.

Thus, based on the comparison of the values in the two inner points, we can replace the current interval $[l, r]$ with a new, shorter interval $[l', r']$. Repeatedly applying the described procedure to the interval, we can get an arbitrarily short interval. Eventually, its length will be less than a certain pre-defined constant (accuracy), and the process can be stopped. This is a numerical method, so we can assume that after that the function reaches its maximum at all points of the last interval $[l, r]$. Without loss of generality, we can take $f(l)$ as the return value.

We didn't impose any restrictions on the choice of points m_1 and m_2 . This choice will define the convergence rate and the accuracy of the implementation. The most common way is to choose the points so that they divide the interval $[l, r]$ into three equal parts. Thus, we have

$$m_1 = l + \frac{(r - l)}{3}$$

$$m_2 = r - \frac{(r - l)}{3}$$

If m_1 and m_2 are chosen to be closer to each other, the convergence rate will increase slightly.

Run time analysis

$$T(n) = T(2n/3) + O(1) = \Theta(\log n)$$

It can be visualized as follows: every time after evaluating the function at points m_1 and m_2 , we are essentially ignoring about one third of the interval, either the left or right one. Thus the size of the search space is $2n/3$ of the original one.

Applying [Master's Theorem](#), we get the desired complexity estimate.

The case of the integer arguments

If $f(x)$ takes integer parameter, the interval $[l, r]$ becomes discrete. Since we did not impose any restrictions on the choice of points m_1 and m_2 , the correctness of the algorithm is not affected. m_1 and m_2 can still be chosen to divide $[l, r]$ into 3 approximately equal parts.

The difference occurs in the stopping criterion of the algorithm. Ternary search will have to stop when $(r - l) < 3$, because in that case we can no longer select m_1 and m_2 to be different from each other as well as from l and r , and this can cause an infinite loop. Once $(r - l) < 3$, the remaining pool of candidate points $(l, l + 1, \dots, r)$ needs to be checked to find the point which produces the maximum value $f(x)$.

Golden section search

In some cases computing $f(x)$ may be quite slow, but reducing the number of iterations is infeasible due to precision issues. Fortunately, it is possible to compute $f(x)$ only once at each iteration (except the first one).

To see how to do this, let's revisit the selection method for m_1 and m_2 . Suppose that we select m_1 and m_2 on $[l, r]$ in such a way that $\frac{r-l}{r-m_1} = \frac{r-l}{m_2-l} = \varphi$ where φ is some constant. In order to reduce the amount of computations, we want to select such φ that on the next iteration one of the new evaluation points m'_1, m'_2 will coincide with either m_1 or m_2 , so that we can reuse the already computed function value.

Now suppose that after the current iteration we set $l = m_1$. Then the point m'_1 will satisfy $\frac{r-m_1}{r-m'_1} = \varphi$. We want this point to coincide with m_2 , meaning that $\frac{r-m_1}{r-m_2} = \varphi$.

Multiplying both sides of $\frac{r-m_1}{r-m_2} = \varphi$ by $\frac{r-m_2}{r-l}$ we obtain $\frac{r-m_1}{r-l} = \varphi \frac{r-m_2}{r-l}$. Note that $\frac{r-m_1}{r-l} = \frac{1}{\varphi}$ and $\frac{r-m_2}{r-l} = \frac{r-l+m_2-m_1}{r-l} = 1 - \frac{1}{\varphi}$. Substituting that and multiplying by φ , we obtain the following equation:

$$\varphi^2 - \varphi - 1 = 0$$

This is a well-known golden section equation. Solving it yields $\frac{1 \pm \sqrt{5}}{2}$. Since φ must be positive, we obtain $\varphi = \frac{1 + \sqrt{5}}{2}$. By applying the same logic to the case when we set $r = m_2$ and want m'_2 to coincide with m_1 , we obtain the same value of φ as well. So, if we choose $m_1 = l + \frac{r-l}{1+\varphi}$ and $m_2 = r - \frac{r-l}{1+\varphi}$, on each iteration we can re-use one of the values $f(x)$ computed on the previous iteration.

Implementation

```
double ternary_search(double l, double r) {
    double eps = 1e-9;           //set the error limit here
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1);       //evaluates the function at m1
        double f2 = f(m2);       //evaluates the function at m2
        if (f1 < f2)
            l = m1;
        else
            r = m2;
    }
    return f(l);                 //return the maximum of f(x) in [l, r]
}
```

Here `eps` is in fact the absolute error (not taking into account errors due to the inaccurate calculation of the function).

Instead of the criterion `r - l > eps`, we can select a constant number of iterations as a stopping criterion. The number of iterations should be chosen to ensure the required accuracy. Typically, in most programming challenges the error limit is 10^{-6} and thus 200 - 300 iterations are sufficient. Also, the number of iterations doesn't depend on the values of l and r , so the number of iterations corresponds to the required relative error.

Contributors:

likecs (39.32%) Yury Semenov (17.09%) Mariia Mykhailova (16.24%) Abhijeet Krishnan (11.11%)
 Oleksandr Kulkov (5.98%) Morass (5.13%) Joaquin Garmendia Cabrera (1.71%) Ahmed Marouf (0.85%)
 Lovish Madhu (0.85%) Puneet Kumar (0.85%) jxu (0.85%)

Original

Simulated Annealing

Simulated Annealing (SA) is a randomized algorithm, which approximates the global optimum of a function. It's called a randomized algorithm, because it employs a certain amount of randomness in its search and thus its output can vary for the same input.

The problem

We are given a function $E(s)$, which calculates the energy of the state s . We are tasked with finding the state s_{best} at which $E(s)$ is minimized. **SA** is suited for problems where the states are discrete and $E(s)$ has multiple local minima. We'll take the example of the [Travelling Salesman Problem \(TSP\)](#).

Travelling Salesman Problem (TSP)

You are given a set of nodes in 2 dimensional space. Each node is characterised by its x and y coordinates. Your task is to find an ordering of the nodes, which will minimise the distance to be travelled when visiting these nodes in that order.

Motivation

Annealing is a metallurgical process, wherein a material is heated up and allowed to cool, in order to allow the atoms inside to rearrange themselves in an arrangement with minimal internal energy, which in turn causes the material to have different properties. The state is the arrangement of atoms and the internal energy is the function being minimised. We can think of the original state of the atoms, as a local minima for its internal energy. To make the material rearrange its atoms, we need to motivate it to go across a region where its internal energy is not minimised in order to reach the global minima. This motivation is given by heating the material to a higher temperature.

Simulated annealing, literally, simulates this process. We start off with some random state (material) and set a high temperature (heat it up). Now, the algorithm is ready to accept states which have a higher energy than the current state, as it is motivated by the high temperature. This prevents the algorithm from getting stuck inside local minimas and move towards the global minima. As time progresses, the algorithm cools down and refuses the states with higher energy and moves into the closest minima it has found.

The energy function $E(s)$

$E(s)$ is the function which needs to be minimised (or maximised). It maps every state to a real number. In the case of TSP, $E(s)$ returns the distance of travelling one full circle in the order of nodes in the state.

State

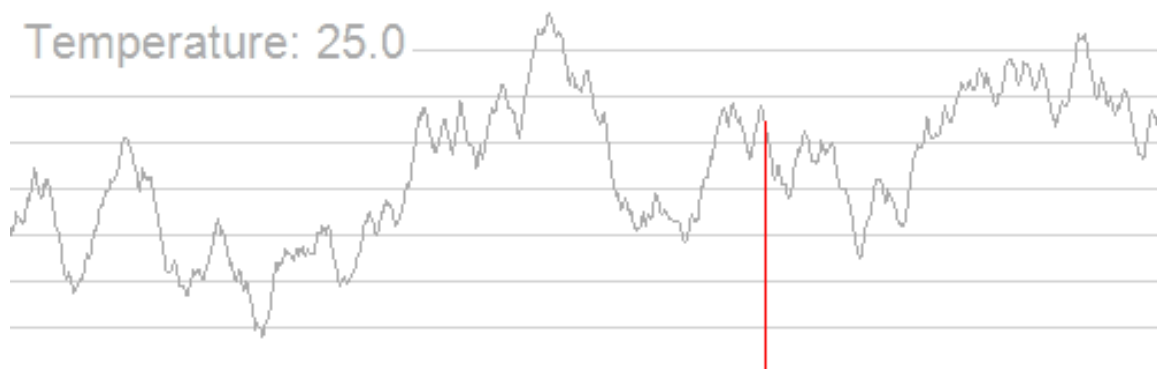
The state space is the domain of the energy function, $E(s)$, and a state is any element which belongs to the state space. In the case of TSP, all possible paths that we can take to visit all the nodes is the state space, and any single one of these paths can be considered as a state.

Neighbouring state

It is a state in the state space which is close to the previous state. This usually means that we can obtain the neighbouring state from the original state using a simple transform. In the case of the Travelling Salesman Problem, a neighbouring state is obtained by randomly choosing 2 nodes, and swapping their positions in the current state.

Algorithm

We start with a random state s . In every step, we choose a neighbouring state s_{next} of the current state s . If $E(s_{next}) < E(s)$, then we update $s = s_{next}$. Otherwise, we use a probability acceptance function $P(E(s), E(s_{next}), T)$ which decides whether we should move to s_{next} or stay at s . T here is the temperature, which is initially set to a high value and decays slowly with every step. The higher the temperature, the more likely it is to move to s_{next} . At the same time we also keep a track of the best state s_{best} across all iterations. Proceeding till convergence or time runs out.



A visual representation of simulated annealing, searching for the maxima of this function with multiple local maxima.

Temperature(T) and decay(u)

The temperature of the system quantifies the willingness of the algorithm to accept a state with a higher energy. The decay is a constant which quantifies the "cooling rate" of the algorithm. A slow cooling rate (larger u) is known to give better results.

Probability Acceptance Function(PAF)

$$P(E, E_{next}, T) = \begin{cases} \text{True} & \text{if } \mathcal{U}_{[0,1]} \leq \exp(-\frac{E_{next}-E}{T}) \\ \text{False} & \text{otherwise} \end{cases}$$

Here, $\mathcal{U}_{[0,1]}$ is a continuous uniform random value on $[0, 1]$. This function takes in the current state, the next state and the temperature, returning a boolean value, which tells our search whether it should move to s_{next} or stay at s . Note that for $E_{next} < E$, this function will always return True, otherwise it can still make the move with probability $\exp(-\frac{E_{next}-E}{T})$, which corresponds to the [Gibbs measure](#).

```
bool P(double E, double E_next, double T, mt19937 rng){
    double prob = exp(-(E_next-E)/T);
    if(prob > 1) return true;
    else{
        bernoulli_distribution d(prob);
        return d(rng);
    }
}
```

Code Template

```
class state {
public:
    state() {
        // Generate the initial state
    }
    state next() {
        state s_next;
        // Modify s_next to a random neighboring state
        return s_next;
    }
    double E() {
        // Implement the energy function here
    };
};

pair<double, state> simAnneal() {
    state s = state();
    state best = s;
    double T = 10000; // Initial temperature
    double u = 0.995; // decay rate
    double E = s.E();
    double E_next;
    double E_best = E;
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    while (T > 1) {
        state next = s.next();
        E_next = next.E();
        if (P(E, E_next, T, rng)) {
            s = next;
            if (E_next < E_best) {
```

```

        best = s;
        E_best = E_next;
    }
    E = E_next;
}
T *= u;
}
return {E_best, best};
}

```

How to use:

Fill in the state class functions as appropriate. If you are trying to find a global maxima and not a minima, ensure that the $E()$ function returns negative of the function you are maximizing and print $-E_{best}$ in the end. Set the below parameters as per your need.

Parameters

- T : Initial temperature. Set it to a higher value if you want the search to run for a longer time.
- u : Decay. Decides the rate of cooling. A slower cooling rate (larger value of u) usually gives better results, at the cost of running for a longer time. Ensure $u < 1$.

The number of iterations the loop will run for is given by the expression

$$N = \lceil -\log_u T \rceil$$

Tips for choosing T and u : If there are many local minimas and a wide state space, set $u = 0.999$, for a slow cooling rate, which will allow the algorithm to explore more possibilities. On the other hand, if the state space is narrower, $u = 0.99$ should suffice. If you are not sure, play it safe by setting $u = 0.998$ or higher. Calculate the time complexity of a single iteration of the algorithm, and use this to approximate a value of N which will prevent TLE, then use the below formula to obtain T .

$$T = u^{-N}$$

Example implementation for TSP

```

class state {
public:
    vector<pair<int, int>> points;
    std::mt19937 mt{ static_cast<std::mt19937::result_type>(
        std::chrono::steady_clock::now().time_since_epoch().count()
    ) };
    state() {
        points = {{0,0},{2,2},{0,2},{2,0},{0,1},{1,2},{2,1},{1,0}} ;
    }
    state next() {
        state s_next;
        s_next.points = points;
        uniform_int_distribution<> choose(0, points.size()-1);
    }
}

```

```

        int a = choose(mt);
        int b = choose(mt);
        s_next.points[a].swap(s_next.points[b]);
        return s_next;
    }

    double euclidean(pair<int, int> a, pair<int, int> b) {
        return hypot(a.first - b.first, a.second - b.second);
    }

    double E() {
        double dist = 0;
        int n = points.size();
        for (int i = 0; i < n; i++)
            dist += euclidean(points[i], points[(i+1)%n]);
        return dist;
    };

    int main() {
        pair<double, state> res;
        res = simAnneal();
        double E_best = res.first;
        state best = res.second;
        cout << "Length of shortest path found : " << E_best << "\n";
        cout << "Order of points in shortest path : \n";
        for(auto x: best.points) {
            cout << x.first << " " << x.second << "\n";
        }
    }
}

```

Further modifications to the algorithm:

- Add a time based exit condition to the while loop to prevent TLE
- The decay implemented above is an exponential decay. You can always replace this with a decay function as per your needs.
- The Probability acceptance function given above, prefers accepting states which are lower in energy because of the $E_{next} - E$ factor in the numerator of the exponent. You can simply remove this factor, to make the PAF independent of the difference in energies.
- The effect of the difference in energies, $E_{next} - E$, on the PAF can be increased/decreased by increasing/decreasing the base of the exponent as shown below:

```

bool P(double E, double E_next, double T, mt19937 rng) {
    double e = 2; // set e to any real number greater than 1
    double prob = pow(e, -(E_next-E)/T);
    if (prob > 1)
        return true;
    else {
        bernoulli_distribution d(prob);
        return d(rng);
    }
}

```

